CS-3180: Comparative Languages Lab Problem 4: Functional Paradigms

PURPOSE: The goal of this assignment is to practice functional programing. You will work with list-based recursive recursion. You will also use local scoping mechanisms in Lisp. You will implement a series of luggage scanning, screening, and filtering functions. To ensure functional programming principles are adapted, imperative looping keywords are banned.

Note: This project establishes the core functions for follow-up project next week. For full credit, these functions must be implemented according to their specifications detailed below.

DIRECTIONS: You will implement a series of functions in a file named **`Contraband.lisp`**. Your program must implement five functions (declared with **`defun`**) matching the specifications detailed below. You are welcome to call functions once they have been written (e.g. **`strip-suitcase`** may call **`remove-item`**).

Constraints:

- **Iterative Constructs**: You are prohibited from using **`loop`**, **`dolist`**, **`dotimes`**, **`do`**, or any other keyword that relies on iterative state changes. Every traversal must be done via list recursion or other non-imperative data procesing manners.

- **Namespace Pollution**: Do not declare helper functions or variables in the global namespace. If a function requires a helper function (such as **`xray`**), you must encapsulate that utility locally using **`labels`** or other keywords.

Required Functions

Implement the following functions according to the specifications below. Be sure to test your functions thoroughly with various inputs.

IS-CONTRABAND

```
(defun is-contraband (contraband-list item) ...)
```

Inputs:

- **`contraband-list`**: A flat list of symbols representing prohibited items (e.g. **`'(WEAPON FIREWORKS ETC)`**).
- **`item`**: A single symbol to check (e.g. **`'FIREWORKS`**).

Output:

- Returns a truthy value if the item in **`contraband-list`**, or **`nil`** if it is not.

COUNT-ITEM

```
(defun count-item (item luggage) ...),
```

Inputs:

- **`item`**: A single symbol to count (e.g. **`'WEAPON`**).
- **`luggage`**: A flat list of symbols representing packed belongings (e.g. **`'(CLOTHES WEAPON BOOKS WEAPON)`**).

Output:

- Returns an integer counting the total number of the item. Return **`0`** if the list is empty

REMOVE-ITEM

```
(defun remove-item (item luggage) ...)
```

Inputs:
- **`item`**: A single symbol to purge from the luggage (e.g. **`'FIREWORKS`**).
- **`luggage`**: A flat list of symbols representing packed belongings.

Output:
- Returns a **flat list** with all matching instances of that symbol filtered out. If the item is not present, returns a list identical to the input **`luggage`**. If **`luggage`** is empty, returns **`nil`**. You will likely want to build and return a new list for this function.

STRIP-SUITCASE

```
(defun strip-suitcase (illegal-items luggage) ...)
```

Inputs:
- **`illegal-items`**: A flat list of symbols containing contraband items to search for.
- **`luggage`**: A flat list of symbols representing packed belongings.

Output:
- Returns a filtered flat list containing only clean items. Returns the original **`luggage`** unmodified if **`illegal-items`** is empty.

XRAY

```
(defun xray (manifest luggage) ...)
```

Inputs:
- **`manifest`**: A flat list of symbols denoting items that **should** be packed in order.
- **`luggage`**: A flat list of symbols representing actual contents in order.

Output:
- A single flat list of booleans (**`T`** or **`NIL`**) stating whether items match index-for-index (e.g. **`'(T NIL T)`**). If the total number of items in **`manifest`** does not equal the total number of items in **`luggage`**, the function must short-circuit and immediately return **`nil`**. See tables below for some examples:

Manifest	'APPLE	'BANANA	'CHERRY	'MANGO	'PEAR
Luggage	'APPLE	'ORANGE	'CHERRY	'APPLE	'PEAR
Return	T	NIL	T	NIL	T

Manifest	'APPLE	'BANANA	'CHERRY	—
Luggage	'APPLE	'BANANA	'CHERRY	'MANGO
Return	NIL (Length mismatch)			

Extra: To verify your implementation, you should construct a test suite at the bottom of your file using **`format`** (or **`print`**) to inspect your outputs. Execute your test suite through the wrapper script:

RUBRIC:

(80 pts) **Base Functionality**

[5] **`is-contraband`** accurately handles lookups and returns correct evaluations

[15] **`count-item`** accurately processes and tallies complete items including duplicates

[20] **`remove-item`** filters targets cleanly and handles absent items and empty lists and functions according to the specifications

[20] **`strip-suitcase`** processes full item collections according to the specifications

[20] **`xray`** handles length mismatches and returns a list of index-boolean according to the specifications

(20 pts) **Style**

[5] Program adheres to looping keyword bans and global function restrictions

[15] Completed lab report in Markdown format

IMPORTANT NOTES:

- **Any submission containing iterative looping keywords (loop, dolist, etc.) will receive an automatic 50% penalty.**
- **Submissions that do not compile will receive a zero**
- **Submissions with improperly cited AI will receive a zero and an academic integrity violation**
- **Submissions that are partially or fully copied from another submission will receive a zero and an academic integrity violation**